City Blocks and Segments Methodology

Prepared by Alejandro Blei

September 2026

City segments are spatial units formed by aggregating contiguous urban blocks. They are designed to balance population across units, penalize morphological dissimilarity within units, and favor more geometrically compact outcomes. This document describes the data and methods used to produce city segments for Juba, South Sudan; Kigali, Rwanda; and Johannesburg, South Africa.

The analysis is organized as three connected workflows. First, zone delineation establishes a persistent spatial framework that constrains later block aggregation. Second, block screening and block splitting identify blocks that meet population, area, or building morphology criteria, splits them, and replaces selected parent blocks with smaller child blocks. Third, block merging aggregates the revised citywide block layer into city segments while balancing population, compactness, and morphological similarity goals. Figure 1 summarizes the three components and shows that the final zones created in the first workflow remain a constraint throughout the later workflows.

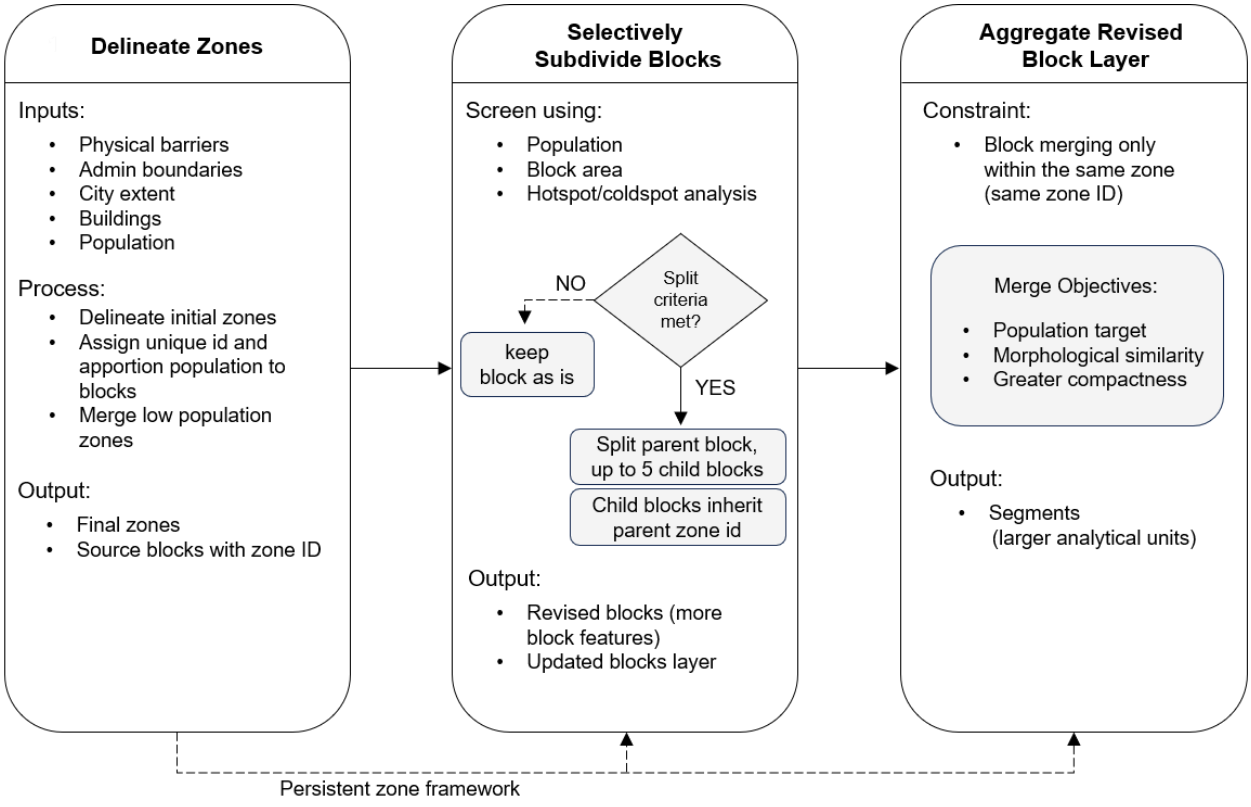


Figure 1. *Overview of the city-segment workflows and the persistent zone constraint*

This document exists alongside two companion project outputs: the City Segments codebase within CIESIN's *World_bank* GitHub repository (https://github.com/ciesin/World_bank/tree/master/city_segments) and the separate readme accompanying the project data. The GitHub repository contains the notebooks and computer code used to implement the analyses described here. The readme serves primarily as a concise reference to the delivered data products.

This document serves a complementary purpose. It translates the computer code into a continuous narrative, explains the rationale and sequence of major analytical steps, and illustrates their application in Juba, Kigali, and Johannesburg through figures, tables, and summary results. Readers seeking to understand or reproduce the analysis should therefore consider the three resources together.

1. Zone Delineation

Overview

The purpose of zone delineation is to partition the city into a stable set of areas that represent meaningful barriers or transitions in the urban landscape. Major roads and waterways act as physical separators; administrative boundaries represent institutional divisions; and a building-density classification distinguishes broad development patterns. Each data source is first processed and translated to the block layer so that every block carries an initial-zone identifier, an administrative identifier, and a density-group value. These three attributes are then combined to define the preliminary analysis zones.

Population data is processed in parallel. A gridded population estimate is apportioned to blocks using building-footprint area as a dasymetric weight, after which block populations are summed to zones. The workflow uses 100 persons as a project-defined minimum zone population. If every preliminary zone meets the threshold, the preliminary layer can serve as the final zone layer; otherwise, low-population zones are consolidated through a sequence of increasingly permissive rules.

The zone delineation workflow is implemented in 14 ordered Jupyter notebooks in the *zones* directory of the City Segments codebase, supplemented by six notebooks in the *knn_buildings* directory that support the building-density classification used in zone construction. Conceptually, the workflow comprises four connected stages: (1) assign zone, administrative, and building density attributes to blocks to create preliminary analysis zones; (2) allocate population to blocks and calculate zone populations; (3) consolidate low-population zones through shared-edge merging; and (4) resolve remaining low population cases, including cross-zone, island, and nearest-feature merging procedures.

Figure 2 follows the workflow from the source datasets through creation of the *zones_2* layer and the first population test. Figure 3 expands the low-population branch, showing the progression from shared-edge merging within initial zones to the treatment of 'islands' (non-contiguous settlement extent) and, finally, nearest-feature merging.

Data Inputs

The zones workflow requires the eight data inputs listed in Table 1. For this analysis, a city's extent and initial block layer were created by methodologies outlined in *GRID3 NGA - Settlement Extents v4.0*[1], and *GRID3 NGA - Settlement Extents v3.1*[2], which the GRID3 team applied upon request to produce the Juba, Kigali, and Johannesburg inputs. Compared to existing off-the-shelf city extent products such as the GHSL Urban Center Database, a distinguishing feature of GRID3 extents is that they may comprise many noncontiguous polygons instead of a single contiguous feature. The three cities' extents comprised 381, 187, and 912 polygons respectively. In each case, a city's extent polygons followed a right-skewed area distribution: few large polygons and many smaller polygons. Initial attempts at zone creation with GRID3 extents revealed that the convex hull of the city extent was the appropriate mask for extracting and processing roads and water data. All other data used in the zones workflow are publicly available and were collected from sources listed in Table 1.

Input	Source	Type
City extent	GRID3	Polygon
City extent hull	Derived from city extent	Polygon
Blocks	GRID3	Polygon
Roads	OpenStreetMap	Line
Water	OpenStreetMap	Line
Admin boundaries	geoBoundaries	Polygon
Buildings	Overture	Polygon
Population	LandScan Mosaic / WorldPop Global 2	Raster

Table 1. *Inputs to the zone delineation workflow*

Roads data was extracted from OpenStreetMap using the osmnx python package; linear water features were downloaded for the respective countries from the Geofabrik platform; administrative boundary data for countries was downloaded from the geoBoundaries Global Database of Political Administrative Boundaries Database, and building data was extracted from Overture Foundation using the overturemaps python package. We assessed population data before deciding which product to use locally, resulting in the selection of Landscan Mosaic (100m) for Juba and WorldPop Global 2 (100m) for Kigali and Johannesburg. Both Landscan and WorldPop websites contain country download features.

Several analysis steps require derived forms. The convex hull of the multipart GRID3 city extent provides a continuous mask for extracting and processing roads and waterways, including the spaces between disconnected settlement polygons. Building footprints are represented by inside points for nearest-neighbor and block-assignment operations.

[1] Center for Integrated Earth System Information (CIESIN), Columbia University. 2026. GRID3 NGA - Settlement Extents v4.0. New York: GRID3. https://doi.org/10.7916/fr96-f546

[2] Center for International Earth Science Information Network (CIESIN), Columbia University. 2024. GRID3 NGA - Settlement Extents v3.1. New York: GRID3. https://doi.org/10.7916/x9xg-e262.

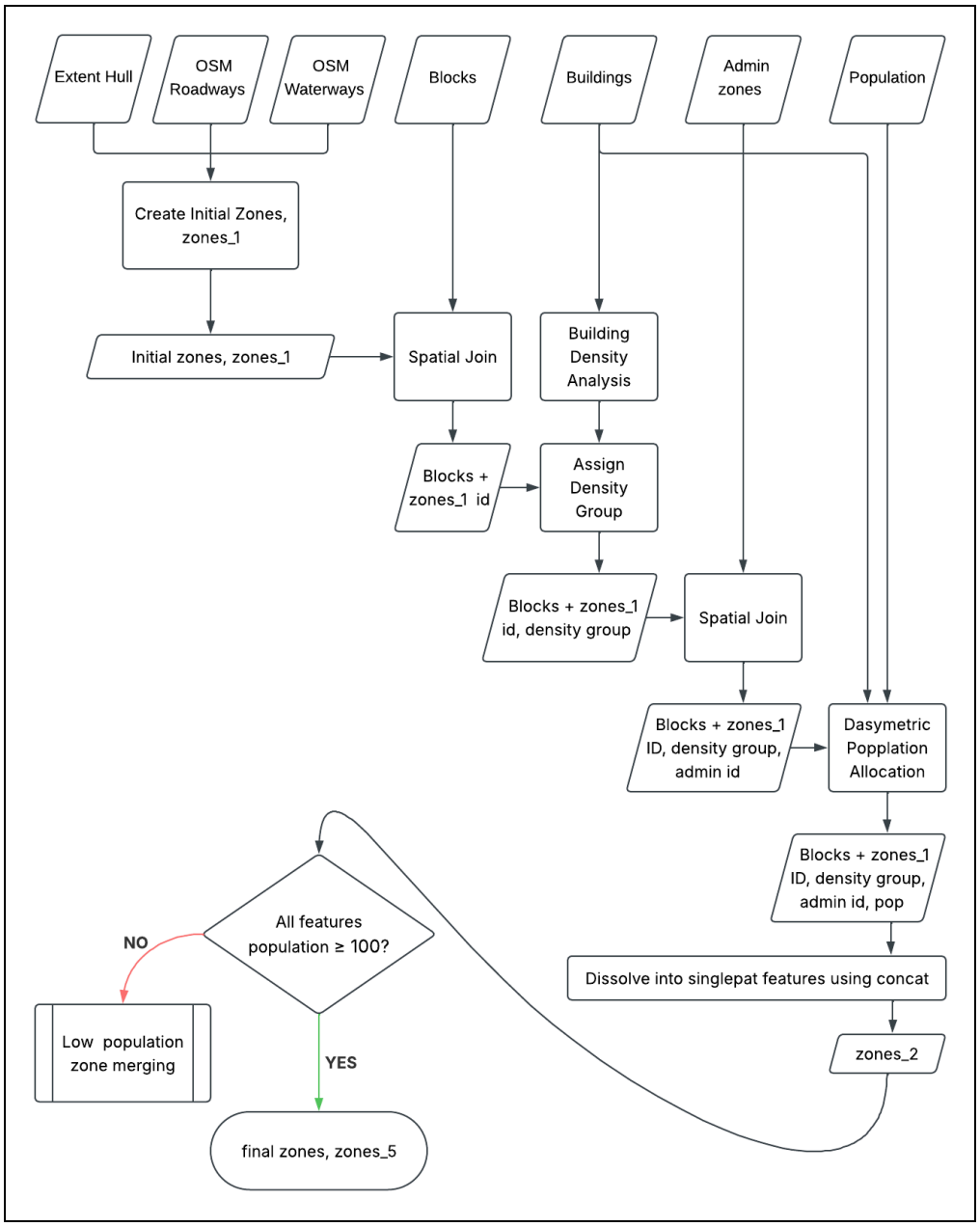


Figure 2. *Zone delineation from source datasets to zones_2 and the population-threshold decision*

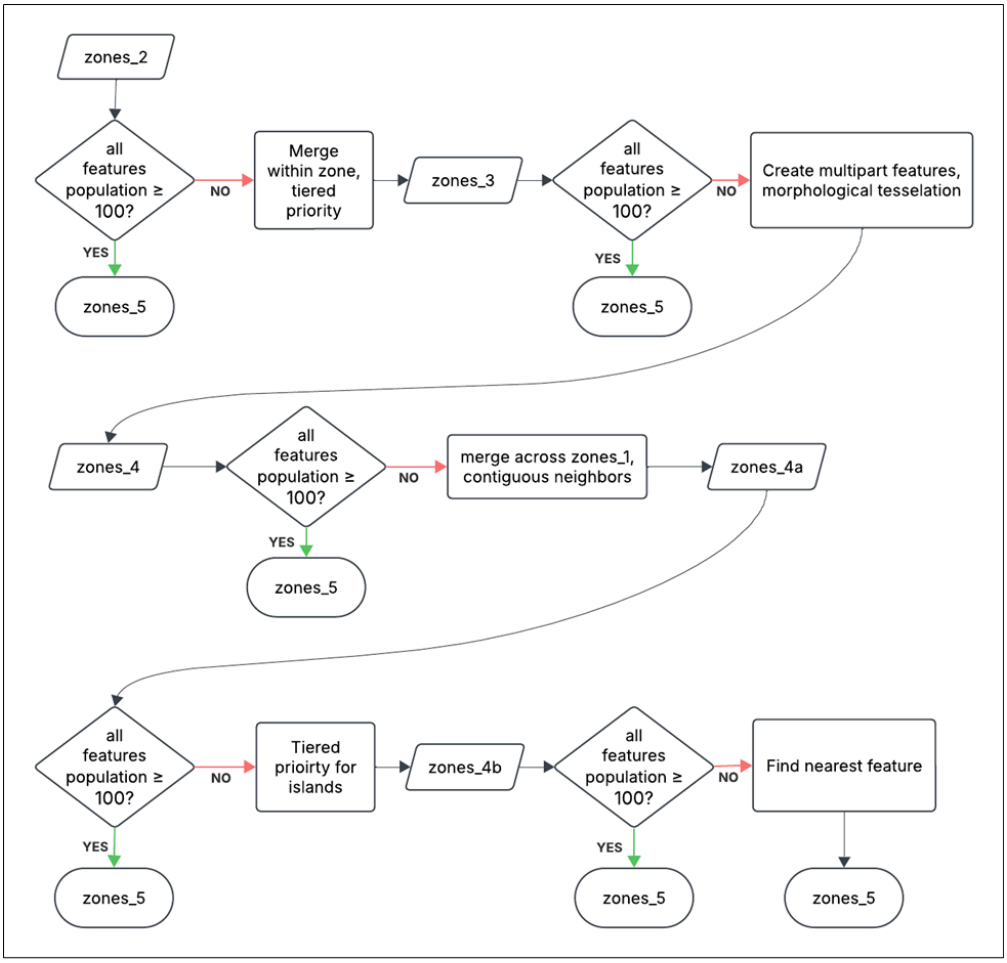


Figure 3. *Tiered consolidation of low-population zones from zones_2 to zones_5*

Create Initial Zones

Initial zones, zones_1, are constructed from the linework that most plausibly separates urban areas. The road branch begins with the drivable OpenStreetMap (OSM) network extracted over the extent hull. Major roads are selected (highway filter = trunk, motorway, or primary), projected, and processed so that dual carriageways, roundabouts, interchanges, and other complex road geometries are represented by appropriate centerlines and junctions rather than by multiple parallel lines. Intermediate polygonization and compactness checks identify narrow artifacts left by roads that did not collapse cleanly. Where appropriate, those artifacts are skeletonized before the road layer is finalized.

OSM waterways are clipped to the extent hull. Administrative polygons intersecting the hull are selected and converted to lines, and the hull is also converted to a boundary line. The processed major roads, waterways, administrative lines, and hull boundary are merged, clipped to the hull, and polygonized. Every enclosed face becomes one zones_1 feature.

The GRID3 blocks do not nest perfectly within zones_1. Instead of cutting the blocks, the workflow creates one representative point constrained to fall inside each block and assigns the

zones_1 identifiers of the polygon containing that point. This preserves the original block geometry while giving every block a consistent categorical membership for the later dissolve.

Figure 4 links the boundary construction to the later block-based result. The left panel shows the processed major roads in red, waterways in blue, and the city extent hull. The right panel previews zones_2, created after zone, administration, and density attributes have been attached to blocks; colors distinguish values of the combined ‘concat’ key.

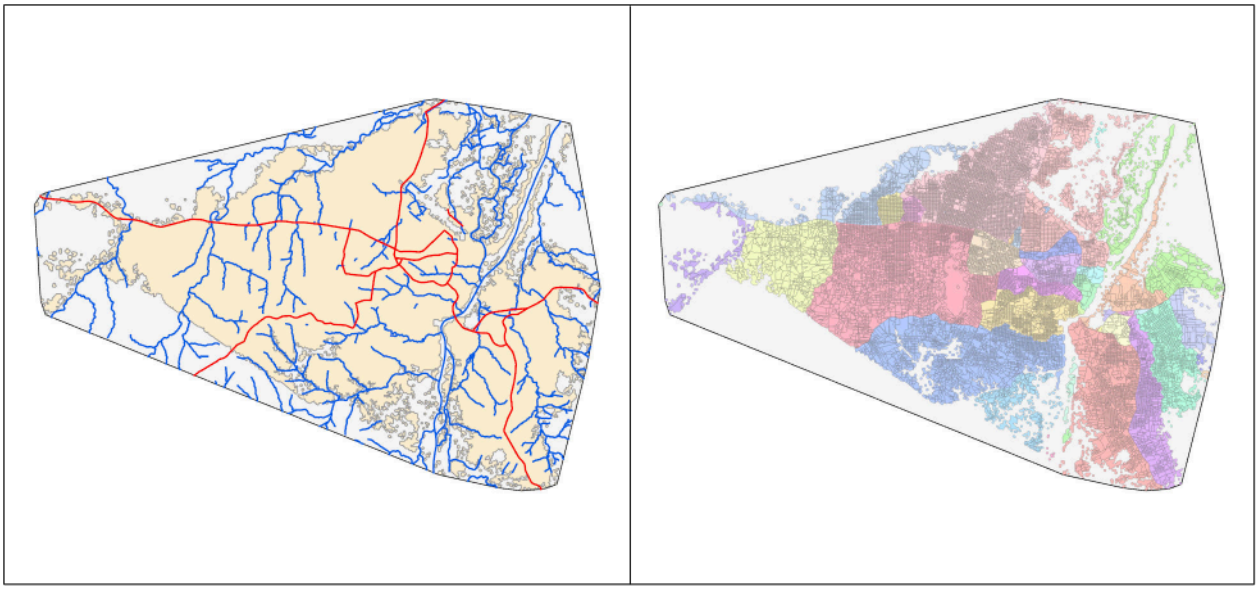


Figure 4. *Boundary inputs and block-based zones_2 in Juba*

Building Density Analysis

Building density is characterized by the spacing of buildings at various distances rather than from a single fixed-radius count. Building footprints are represented by points, and the distance from each building to its kth nearest neighbor building is measured at several ranks. The local analysis uses the 5th, 10th, 20th, 40th, and 80th neighbors. A broader analysis extends the range to the 200th, 500th, 1,000th, 2,000th, 4,000th, and 8,000th neighbors. Small ranks describe immediate spacing; larger ranks describe how a building is embedded in a neighborhood and the broader urban fabric.

Distances are explored with distribution plots, empirical cumulative distributions, boxplots, maps, and local-to-broad distance ratios. Ratios are important because two buildings can have similar immediate spacing but very different surroundings at larger scales. For example, a low distance to the 20th neighbor combined with a high distance to the 1000th neighbor indicates a compact local cluster within a discontinuous neighborhood. Before clustering, selected distances and ratios are log-transformed to reduce skew and standardized so that variables measured on different scales contribute comparably.

K-means models are evaluated for candidate values of k from 2 through 6. Elbow plots, silhouette scores, principal-component views, mapped patterns, and cluster-profile tables support the choice; no single diagnostic selects k automatically. The analyst must also judge whether the classes are spatially coherent and useful for the zone framework. In the three city applications, a two-class broad-scale model was selected, producing one density-group label

per building. Because K-means class numbers are nominal, the cluster profiles must be inspected before interpreting or harmonizing the labels as relatively denser or sparser development.

The building classification is transferred to blocks by assigning each building point to a block and taking the majority class among buildings in that block. If the two classes are tied, shared boundary length with already classified neighboring blocks is totaled by class, and the class with the greatest total shared edge is recommended. Blocks with no buildings, unresolved ties, or no usable classified edge-neighbor receive a documented default value, which in the current implementation is the lower-density class. The resulting cluster_revised field supplies the density component of the concat key.

Figure 5 illustrates why multiple neighbor ranks are informative. The left example is dispersed: even the nearest buildings are separated, and the ring of neighbors expands unevenly across open space. The right example is compact and regular, with successive neighbors encountered over much shorter distances. Two locations could look similar for the first five neighbors yet diverge by the 80th neighbor, which is precisely the distinction captured by the multiscale metrics.

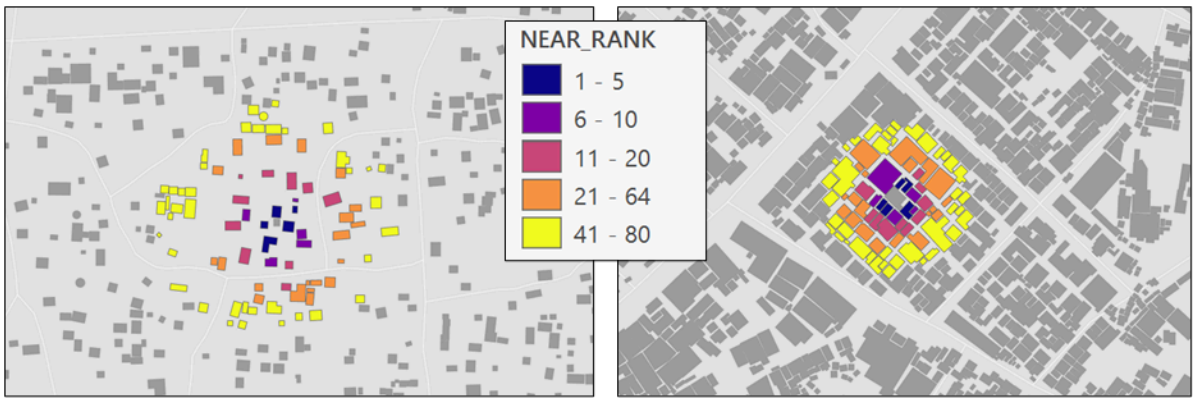


Figure 5. *Nearest-neighbor building configurations in dispersed and compact settings*

Population Apportionment

Source population data is raster grid cells, but the analysis requires population by block. Direct assignment is not possible because grid and block boundaries do not line up. Simple areal interpolation would assign a cell's population according to the proportion of the cell overlapping each block, implying that residents are distributed uniformly across the cell. This workflow uses a building-weighted dasymetric allocation, treating observed building-footprint area as evidence of how population is likely to be distributed.

The population raster is clipped to a rectangular envelope around the settlement extent, converted to points, and rebuilt as a vector fishnet that preserves each cell's population value and identifier. Cells intersecting the extent are retained. For cells that straddle the settlement boundary, Overture buildings are also downloaded for the portion outside the extent.

For each grid cell-block intersection area, the workflow sums the building area inside. It then divides that value by the total building area across the full grid cell. The resulting building-area

share is multiplied by the grid-cell population. Finally, all allocated pieces associated with a block are summed to obtain the block population.

Figure 6 shows the logic for population cell 33063, which overlaps blocks 4927 and 4833. A purely areal method might divide the population between the two blocks according to their overlap areas. In the illustration, however, all highlighted building area within the cell lies in block 4927. The dasymetric weight therefore assigns the cell's population to block 4927 rather than to the larger but unoccupied overlap in block 4833. Repeating this calculation for every gridcell-block intersection area and summing by block produces the population field used in zone construction.

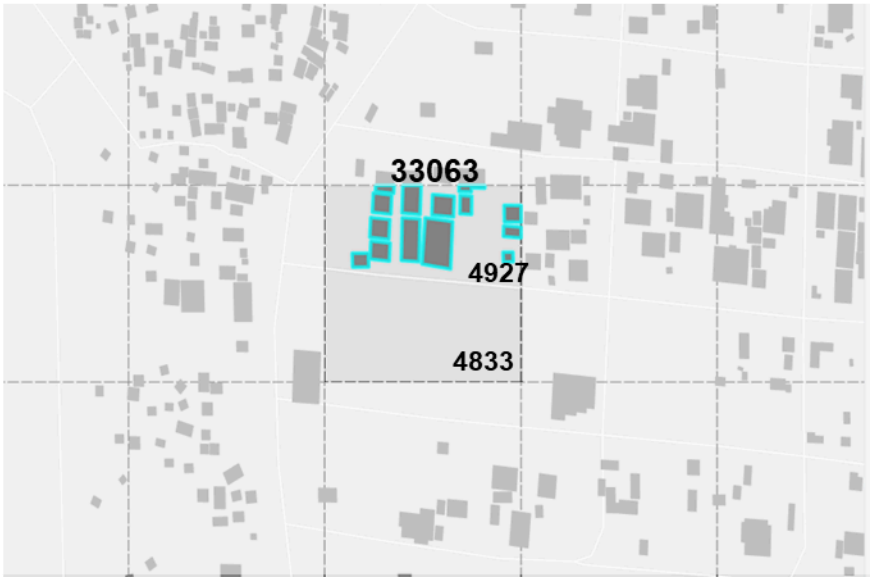


Figure 6. *Building-weighted dasymetric allocation of population cell 33063*

Create zones_2

Each block now carries three categorical attributes: the zones_1 identifier, the administrative identifier, and cluster_revised, the block's density group. The values are concatenated into a single text key, concat. Blocks with the same three-part key are dissolved, but the output is explicitly single-part. Consequently, disconnected groups that share the same concat value remain separate features rather than becoming one multi-part record.

Population is not summed directly as a dissolve statistic because that would summarize the entire concat group and could assign one total to multiple disconnected single-part features. Instead, each zones_2 feature receives a stable zone2_id. Inside points from the original blocks are joined to zones_2, and block population and block count are summarized by that identifier. Quality checks compare the input and output population totals, confirm that every block point was assigned, and distinguish the number of unique concat values from the larger number of possible single-part features.

The first stopping test is then applied. If every zones_2 feature has a population of at least 100, no consolidation is required and zones_2 can be finalized as zones_5. If one or more features

fall below the threshold, the workflow proceeds to the tiered low-population procedure. Figure 7 illustrates zones_2 after this single-part dissolve; colors distinguish single-part ‘zone-administration-density’ features.

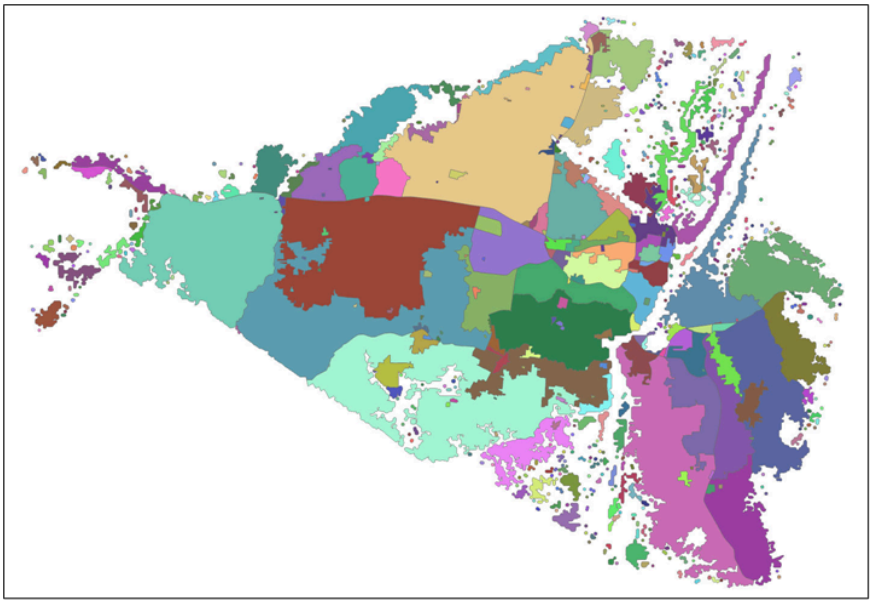


Figure 7. *Juba zones_2 after the single-part dissolve of the zone-administration-density key*

Low-Population Zone Workflow

Figure 8 highlights the zones_2 features that remain below 100 persons. Many occur at the urban periphery, where settlement polygons are small or fragmented, but low-population pieces can also occur along major roads, waterways, administrative boundaries, or transitions between density groups. Merging all of them by the same nearest-neighbor rule would quickly erase the distinctions that the zone framework was designed to preserve. The workflow therefore begins with strict rules and relaxes them only when necessary.

The first consolidation stage creates zones_3 and never crosses the original zones_1 boundary. Low-population features are processed from the smallest population upward using only neighbors that share edges. Eligible neighbors are considered in four tiers: (1) same administrative area and same density group; (2) different administrative area but same density group; (3) same administrative area but different density group; and (4) different administrative area and different density group. The first tier containing at least one candidate is used. This ordering gives priority to retaining the density class, while still preferring the administrative class when both can be preserved.

If a tier contains only one eligible neighbor, the merge is direct. If it contains multiple eligible neighbors, the workflow tests the geometry produced by merging with each candidate and selects the result with the highest Reock compactness. Higher values favor a compact recipient rather than a long or irregular one. After every merge, the geometry and shared-edge neighbor table are rebuilt because the set of eligible relationships may have changed. The recipient survives, population and block counts are summed, and pipe-delimited histories preserve the absorbed zone, administration, and density values in a merge log.

The within-zone process stops when all features meet the population threshold or no further eligible same-initial-zone merge is available. The remaining cases are typically islands with no shared-edge neighbor, or pieces for which satisfying the threshold requires crossing an initial-zone boundary. These cases are handled in the next stages.

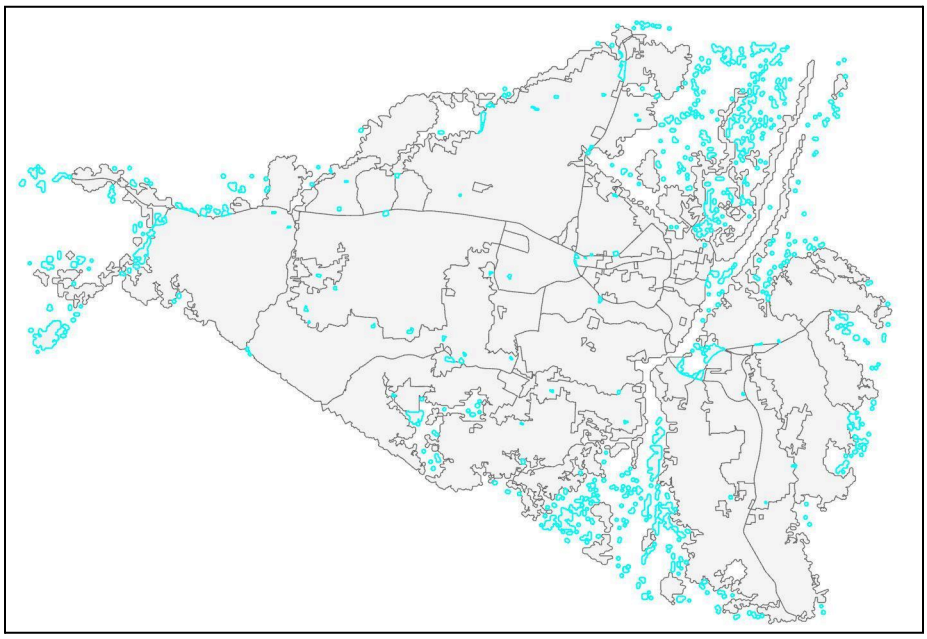


Figure 8. *Low-population zones_2 features requiring consolidation in Juba*

Morphological Tessellation

An island is a zone feature with no shared-edge neighbor; where the source settlement extent includes many detached peripheral polygons or where narrow separations interrupt ordinary polygon adjacency. Because polygon neighbors tables describe shared boundaries, they cannot provide a meaningful merge candidate for an island. The workflow therefore adapts a morphological tessellation function from the momepy python package to construct auxiliary boundaries around zones_3 features.

Morphological tessellation partitions the hull into one tessellation cell per source feature. The extent hull defines the analysis area. Representative points constrained to fall inside the source features provide validation that every feature was associated with a tessellation cell.

Tessellation cells are then dissolved as single-part polygons by concat. Each zones_3 representative point is assigned to one of these dissolved tessellation regions, and zones_3 features sharing a region are then dissolved as a multi-part feature. Population and block counts are summed to create zones_4. This permits spatially separated pieces with the same underlying zone-administration-density identity to be treated together. Figure 9 contrasts the tessellation at left with the simplified grouping it supports at right.

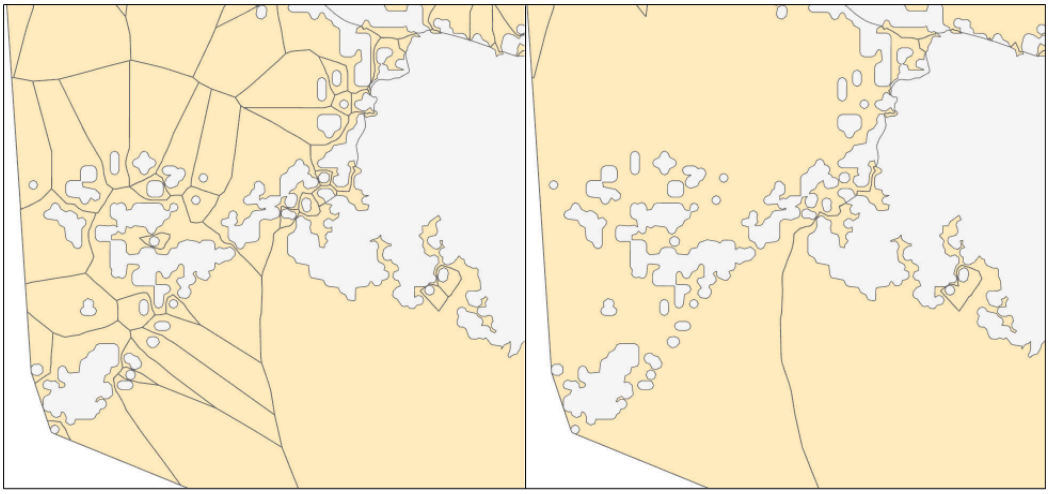


Figure 9. *Morphological tessellation used to associate isolated zone features*

Final Zones: zones_5

Some zones_4 features may still remain below 100 persons. Remaining steps broaden the admissible relationships only after the preceding options have been exhausted. Throughout these steps, geometry is unioned, population and block counts are summed, and the source identifiers and categorical histories are appended to pipe-delimited tracking fields.

Cross-zone shared-edge merges (zones_4a). The workflow first considers low-population features that share an edge with a feature from a different original zone. Candidates are prioritized as: same administration and same density; different administration but same density; and different density. Within the first available tier, the candidate with the longest shared boundary is selected. This stage allows an initial-zone boundary to be crossed, but only where the source and recipient are physically contiguous.

Island consolidation (zones_4b). Next, the focus turns to low-population features with no shared-edge neighbors are treated as islands. The preferred non-island target has the same primary concat value. If none exists, the workflow searches within the same original zone for a feature with a different density group, preferring the same administrative area before a different administrative area. Within each eligible tier, the target with the largest population is selected. These merges can produce multipart zones; the tessellation and shared attributes establish the relationship even though the pieces do not touch.

Last-resort nearest-feature merges (zones_5). Any feature still below the threshold is merged with the nearest remaining feature. Distance is measured from the source feature's convex hull to a candidate's edge, reducing distortions caused by complex or multipart source shapes. Distance ties are resolved in favor of the candidate with the larger population and then the lower feature identifier.

The process continues until no feature remains below the threshold, producing zones_5. A final quality review confirms population conservation, inspection of merge logs, verification that every feature meets the threshold, and review of multi-part or long-distance merges created late in the sequence. The completed zones_5 layer shown in Figure 10 becomes the persistent zone framework: its identifier is transferred to blocks and later constrains block merging so that final city segments do not cross zone boundaries.

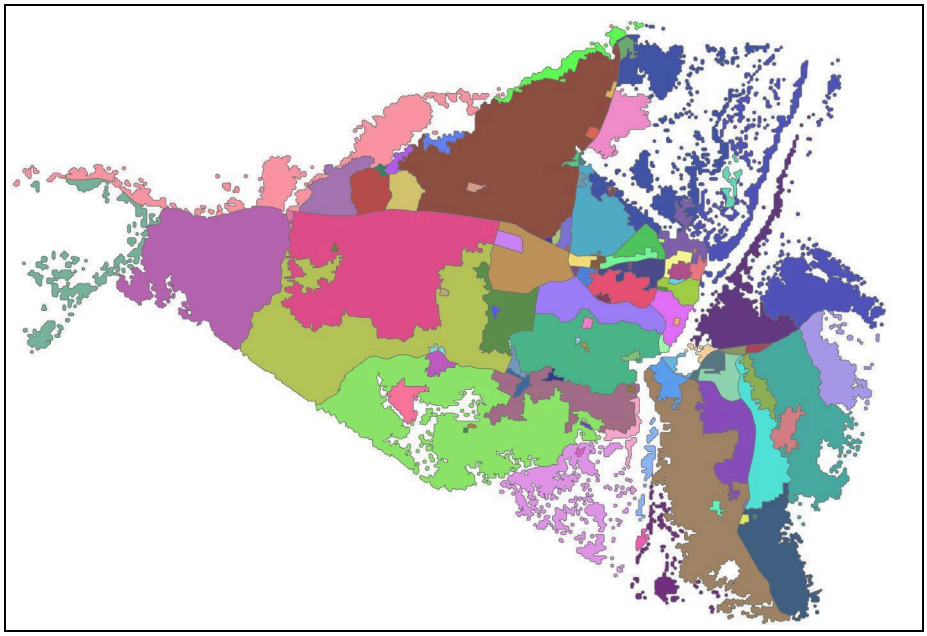


Figure 10. *Final Juba zones (zones_5) after low-population consolidation*

2. Block Screening and Block Splitting

Overview

The second workflow modifies the citywide block layer selectively. It identifies the comparatively small set of blocks whose population, area, or internal building pattern makes them undesirable for the aggregation that occurs in workflow 3. Each of the selected ‘parent’ blocks is replaced by two or more child blocks, while unselected blocks retain their original geometry and attributes. Because the children inherit the parent's zone identifier, subdivision does not weaken the zone framework established in Section 1.

The workflow is implemented in 15 ordered Jupyter notebooks in the *block_splitting* directory of the City Segments codebase. Conceptually, these notebooks form five connected stages: (1) screen blocks; (2) construct building-level spatial units and context variables; (3) generate and refine spatially contiguous cluster solutions; (4) assign population and choose a preferred subdivision; and (5) finalize the child geometries before replacing their parents in a copy of the citywide block layer. An alternative ‘beam’ procedure is available for selected blocks whose building pattern is not satisfactorily divided by the primary clustering approach. Figure 11 illustrates the workflow components and the interactions among inputs, outputs, and processes. These are described in more detail below.

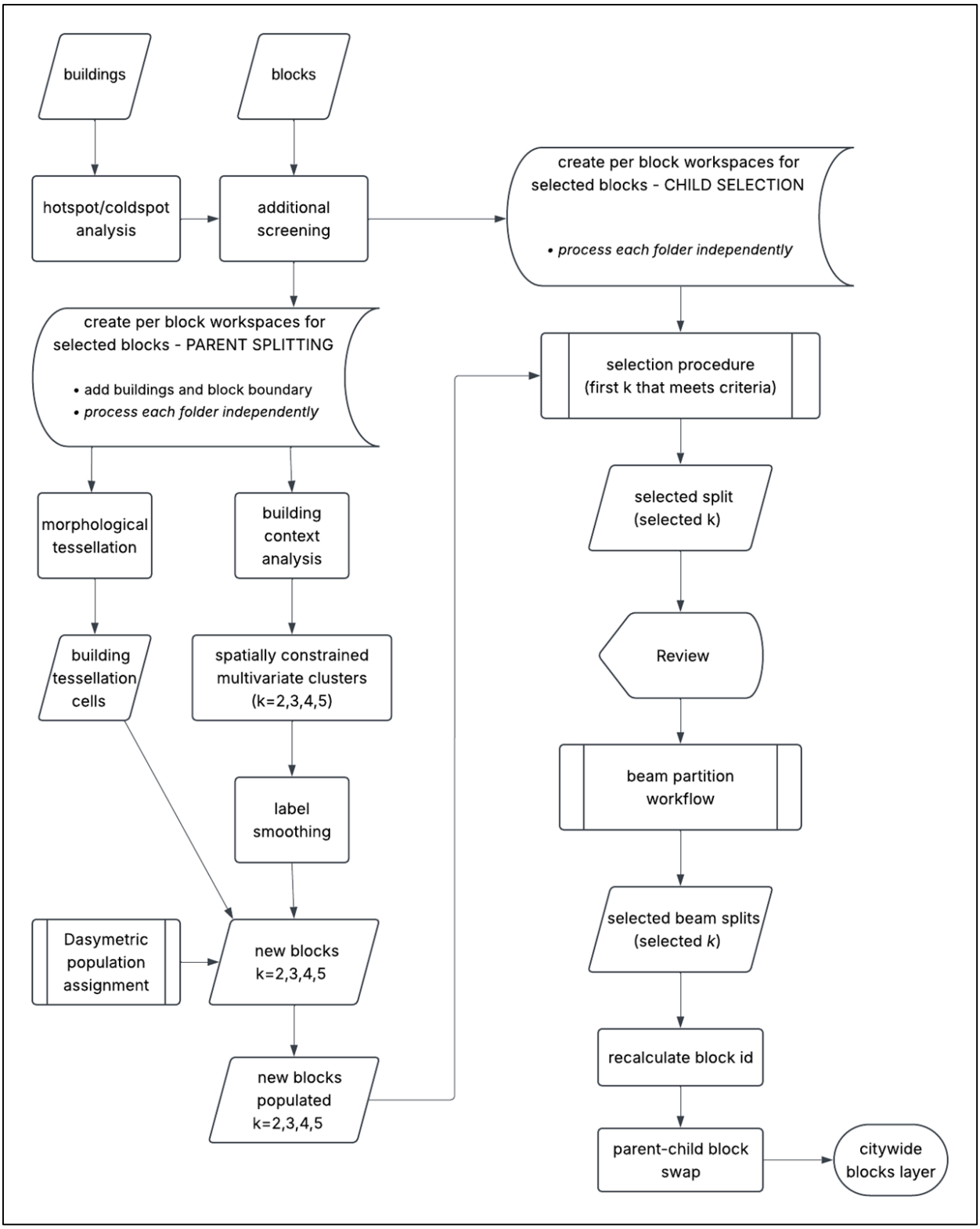


Figure 11. *Workflow diagram for block screening, splitting, child selection, and block swapping.*

Screening Blocks for Subdivision

Screening begins by identifying statistically significant spatial clusters of high and low building footprint values using the ArcGIS Optimized Hotspot Analysis tool. The resulting Gi_Bin values identify statistically significant clusters of relatively large buildings (positive values) and relatively small buildings (negative values). Each analyzed building is represented by an inside point and assigned to a block. Counts of buildings in all seven Gi_Bin classes, from -3 through +3, are then summarized to the block layer. The most stringent classes, Gi_Bin = +3 and Gi_Bin = -3, represent hot spots and cold spots at the 99 percent confidence level and supply the building-morphology criteria used for selection.

> **HH_CC.** The block contains at least one hot-spot building and at least one cold-spot building. The coexistence of both extremes provides evidence of internal morphological heterogeneity.
>
> **HH_Grtr10ha.** The block contains one or more hot-spot building(s), contains no cold-spot building, and is larger than 10 hectares.
>
> **CC_Grtr10ha.** The block contains one or more cold-spot building(s), contains no hot-spot building, and is larger than 10 hectares.

These three heterogeneity flags are mutually exclusive. A separate large population (LargePop) flag identifies blocks with population greater than 1,000 and may overlap any heterogeneity flag. A block enters the splitting workflow when at least one of the four flags is positive. Table 2 distinguishes LargePop and non-LargePop selections and then disaggregates each group by the applicable building-morphology flag.

Screening composition	Juba	Kigali	Johannesburg
Overall composition			
LargePop	19	91	12
Not LargePop	80	158	246
Total	99	249	258
LargePop breakdown			
LargePop only	6	69	8
LargePop and HH_CC	1	1	0
LargePop and HH_Grtr10ha	12	20	4
LargePop and CC_Grtr10ha	0	1	0
Total	19	91	12
Not-LargePop breakdown			
Not LargePop and HH_CC	5	2	3
Not LargePop and HH_Grtr10ha	75	153	243
Not LargePop and CC_Grtr10ha	0	3	0
Total	80	158	246

Table 2. *Composition of blocks selected for splitting by screening criterion and city*

Most selected blocks were not selected because of high population. Across the three cities, 484 of 606 selected blocks (79.9 percent) were not LargePop. The dominant category was Not LargePop and HH_Grtr10ha: 75 blocks in Juba, 153 in Kigali, and 243 in Johannesburg, or 471 blocks in total. Thus, the workflow is primarily responding to large blocks that contain localized building hot spots, with the population rule adding a smaller but important group of densely occupied blocks.

Building Tessellation and Context Variables

Each selected block receives its own workspace containing a copy of the parent block and the citywide building footprints clipped to that block. Organizing outputs by parent block keeps the alternative subdivisions, diagnostic tables, and final selection together and allows a failed or manually reviewed case to be rerun without repeating the entire citywide workflow.

A building-level morphological tessellation (like the tessellation in workflow 1) partitions the parent block into one cell for each building. The tessellation is bounded by the block geometry and linked to buildings through a stable context_bldg_id. In the implementation, source footprints are slightly contracted and their boundaries densified before the momepy tessellation is created; the current settings use a 0.4 m inward offset and 0.5 m segment spacing. The resulting cells provide a continuous polygon fabric that can later be dissolved into child blocks.

A building-context analysis describes the focal building and its relationships to surrounding buildings. For each building, cell area is calculated from its tessellation cell, and building coverage ratio is calculated as building footprint area divided by cell area. Neighbor relationships use exact building polygon-edge distances rather than centroid distances. Candidate neighbors are drawn from the citywide building layer, so the context for buildings in a parent-block boundary is not artificially truncated within that boundary. Once a building falls within a fixed distance, its complete footprint area contributes to neighborhood summary calculations.

The clustering variables represent building size, spacing, and surrounding development at several scales. They include log building area; building coverage of the tessellation cell; edge distance to the 5th, 10th, and 40th nearest buildings; mean log neighbor area for the nearest 5, 10, and 40 buildings; and building counts and total building area within 30 m, 60 m, and 100 m. Additional nearest-neighbor measures are retained for diagnostics, but this selected set supplies the multivariate clustering model. The building level context variables are input to the next stage.

Spatially Constrained Clustering and Label Smoothing

The building clustering method uses the ArcGIS Spatially Constrained Multivariate Clustering tool, which implements the SKATER graph-partitioning method. A trimmed Delaunay triangulation defines which buildings may be grouped as spatial neighbors. SKATER partitions that graph using the building-context variables while preserving spatial connectivity, avoiding geographically disconnected groups that an unconstrained multivariate clustering method could produce.

Four candidate solutions are generated for each parent block, with k = 2, 3, 4, and 5 clusters. Each cluster must contain at least 10 buildings, so a value of k is skipped if the block has fewer than 10 complete building records. The tool runs 100 permutations and, where available, returns a membership probability for each building. No single k is selected at this stage; all feasible solutions are retained because the preferred subdivision is chosen later from child-block population and area.

Figure 12 shows the four alternatives for one selected block, proceeding clockwise from k = 2 in the upper-left panel to k = 5 in the lower-left panel. The partitions remain spatially coherent as additional groups are introduced, and the new groups correspond to recognizable changes in building pattern rather than isolated, spatially dispersed labels.

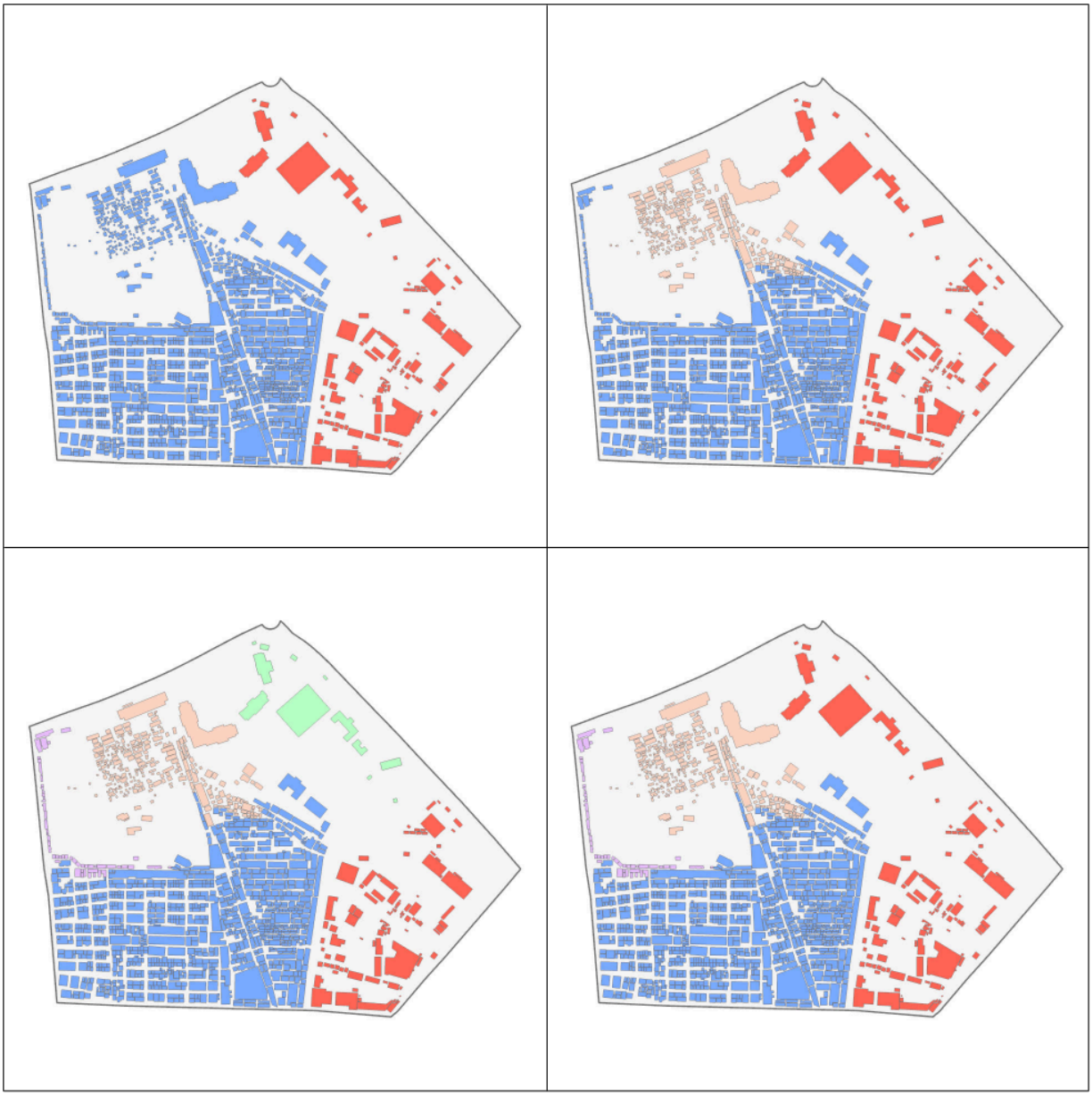


Figure 12. *SKATER building-cluster solutions for k = 2 through 5, clockwise from upper left*

Even with a spatial constraint, a small number of building labels can be locally inconsistent. Label smoothing builds an undirected neighborhood graph where buildings are neighbors when their polygon edges are no more than 30 m apart. Every neighbor building receives equal weight. A building is relabeled when its original cluster accounts for no more than 15 percent of its neighbors and the strongest alternative accounts for at least 85 percent. A second, more

permissive rule applies when the original share is no more than 25 percent, the alternative share is at least 75 percent, and the original membership probability is no greater than 0.50.

The original label, smoothed label, local shares, membership probability, and reason for any change are retained for review. Figure 13 illustrates a relabeling example. In the left panel, three small red buildings lie adjacent to the predominantly blue group at the far left. The neighborhood evidence assigns those buildings to blue in the right panel, shifting the hypothetical block split (thick dissolved boundary) while leaving the broader two-group structure intact.

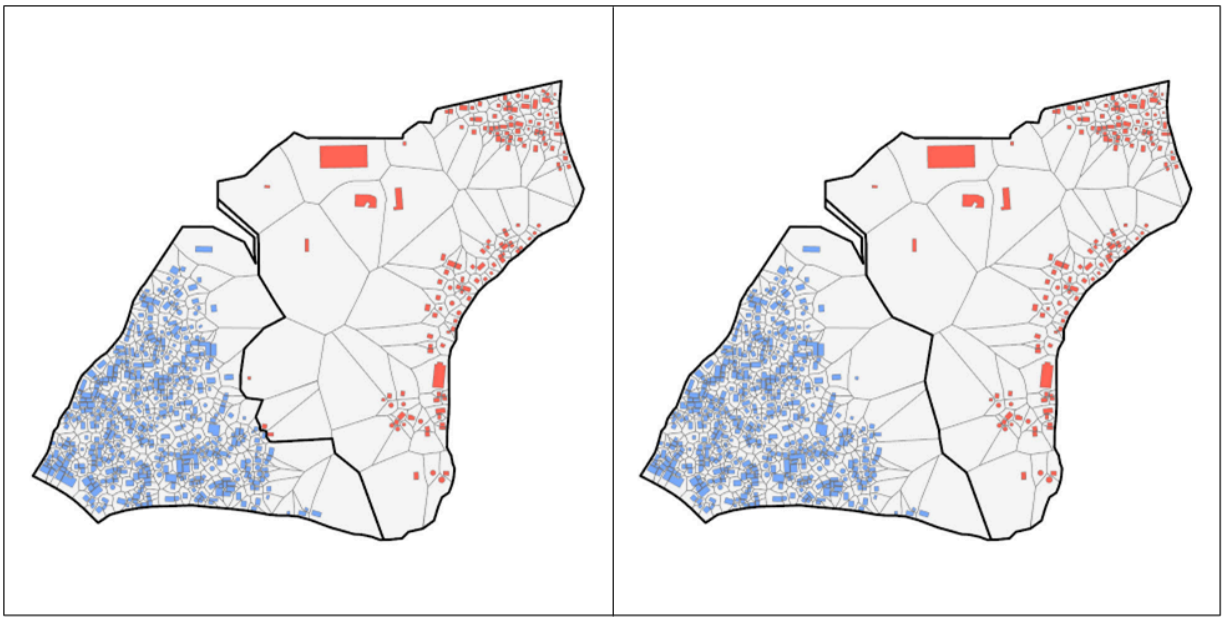


Figure 13. *Building-cluster groups before (left) and after (right) 30 m neighborhood smoothing*

Creating and Selecting Candidate Child Blocks

The smoothed building label is transferred to the corresponding tessellation cell using context_bldg_id. For each available k, cells with the same cluster_smooth value are dissolved, translating the building classification into candidate child-block polygons. Building footprint area and tessellation-cell area are summed for each child. Disconnected pieces with the same label may initially form a multipart candidate. Multi-part geometries are converted to single-part in a final step, but resolving their underlying cause merits attention in future versions of this work. It is almost certainly related to a combination of block shape and the Delaunay triangulation.

Population is assigned to every candidate k solution using the same building-weighted dasymetric principle described in workflow 1. Population-grid cells and buildings intersecting the parent-block area are selected. Within each grid cell, the share of building footprint area falling in each candidate child is multiplied by the grid-cell population, and allocated values are summed by child block. Grid-apportionment tables preserve the building-area denominator, child share, allocated population, and any population associated with building area outside the candidate polygons.

Large-population parents. The workflow checks k = 2, 3, 4, and 5 in order and selects the first solution in which every child has population below 1,000. Choosing the smallest satisfactory k avoids creating more blocks than are needed to meet the population objective.

Heterogeneous parents that are not LargePop. The workflow selects the first k for which every child has tessellation-cell area below 100,000 m^2 (10 hectares). The summed cell_area_m2 field is used because the candidate child's full spatial area is the relevant measure.

If none of the available alternatives satisfies the primary rule, k = 5 is used as the standard fallback and the exception is recorded Missing candidate layers, null population or area values, and threshold violations are written to the selection log and summary rather than silently accepted. The chosen layer is copied to a separate selection directory, while the unselected alternatives remain in the parent workspace for review.

Alternative Beam Splitting

Some selected blocks contain buildings that are tightly packed and lack clear clustering patterns. For these cases, the primary SKATER solutions may satisfy a numerical threshold without creating a convincing internal boundary. A pre-beam diagnostic buffers each building by 2 m, 3 m, and 4 m, dissolves overlapping buffers into singlepart groups, and joins the resulting group counts to the parent's population, area, and building-area measures. The diagnostic table and visual inspection guide a manual decision about which parents should enter the alternative workflow.

Beam treats the building tessellation as a graph whose nodes are cells and whose edges are shared boundary segments. A bounded beam search generates candidate partitions with n = 2, 3, and 4 groups. Every final group must be graph-contiguous. Building footprint area is the balancing weight: for n = 2, each group must contain 40-60 percent of the parent's total building area; for n = 3 or n = 4, each group must contain 18-45 percent. Within those bounds, the objective favors short, simple internal boundaries and penalizes additional cut edges, diagonal boundary segments, and severe building-area imbalance.

The selected tessellation groups are dissolved to create beam candidate blocks. Population is apportioned to each n solution by building-area share, just as it is for the SKATER-derived candidates. LargePop parents use the lowest n for which every child has population below 1,000; non-LargePop heterogeneous parents use the lowest n for which every child has cell_area_m2 below 100,000 m2. If no candidate passes, n = 4 is retained with a warning. The standard selection is preserved as a pre-beam backup before the chosen beam layer becomes the canonical child-block file.

Figure 14 shows n = 2, 3, and 4 candidates for a high-population block whose buildings are close together and do not form discernible clusters. The beam boundaries divide the tessellation into contiguous pieces without relying on weak differences in building context. The final choice is the lowest n at which all child populations fall below 1,000 (in this case n=2).



Figure 14. *Beam candidate subdivisions for n = 2, 3, and 4 in a densely built, high-population block*

Final Child Blocks and Citywide Replacement

Each selected child receives a lineage-preserving identifier of the form blk_<parent>_<subdivision>_<LargePop>_<heterogeneity>_<beam>. The LargePop code is binary; the heterogeneity code records a no heterogeneity flag (0), HH_CC (1), HH_Grtr10ha (2), or CC_Grtr10ha (3); the beam code records whether the standard clustering method (0) or BEAM (1) produced the subdivision. This structure makes the parent and selection pathway recoverable from the identifier itself.

Before citywide replacement, outputs are screened for multipart child geometries. A multi-part feature that is completely surrounded by another child is absorbed into that surrounding feature. Remaining multi-part children are exploded into separate children and receive suffixed identifiers. Because these operations change geometry, cell-area, building-area, cluster, and population values that are no longer valid are recalculated and population is recalculated from the grid and building inputs.

The final swap is performed on a copy of the master citywide blocks layer. Multipart-resolved outputs take priority where they exist; otherwise, the selected child layer is used. Children are inserted before the original parent is deleted so that an insertion failure does not remove the source feature. Two binary fields provide simple citywide lineage: was_split equals 1 for a child and 0 for an untouched original block, while beam equals 1 only for children created by the alternative method.

Citywide count	Juba	Kigali	Johannesburg
Input blocks	19,438	16,080	58,527
Blocks selected for splitting	99	249	258
Final blocks after splitting	19,668	16,620	59,034

Table 3. *Citywide block counts following the block-splitting workflow*

The screening step selected only 99 of 19,438 input blocks in Juba (0.51 percent), 249 of 16,080 in Kigali (1.55 percent), and 258 of 58,527 in Johannesburg (0.44 percent). Across the three cities, 606 of 94,045 input blocks were selected, or 0.64 percent. Replacing those parents with their children increased the combined block count by 1,277, from 94,045 to 95,322. The method therefore introduces finer geometry where it is analytically justified without broadly altering the citywide block dataset.

3. Block Merging and City Segments

Overview

The third workflow aggregates the revised citywide block layer produced in Section 2 into larger units called city segments. The zones_5 geography developed in Section 1 remains an organizing constraint throughout this process: blocks assigned to different zones are never merged into the same city segment. Within each zone, the workflow seeks to form segments with populations generally between 500 and 1,000 people while favoring similar patterns of building development and more geometrically compact shapes.

The workflow is implemented in four ordered Jupyter notebooks in the *block_merging* directory of the City Segments codebase. The first notebook prepares the block, zone, and building inputs. The second notebook calculates building statistics for each block. The third notebook evaluates whether small geometric inconsistencies affect the block-neighbor relationships needed for merging. The fourth notebook performs the block aggregation itself in two steps: first by merging blocks that share boundaries within the same zone, and then by addressing remaining low-population segments that could not be resolved through contiguous block merging. Blocks classified as open space or airport are excluded from both merging steps and remain standalone units. Figure 14 provides an overview of the main steps associated with workflow 3.

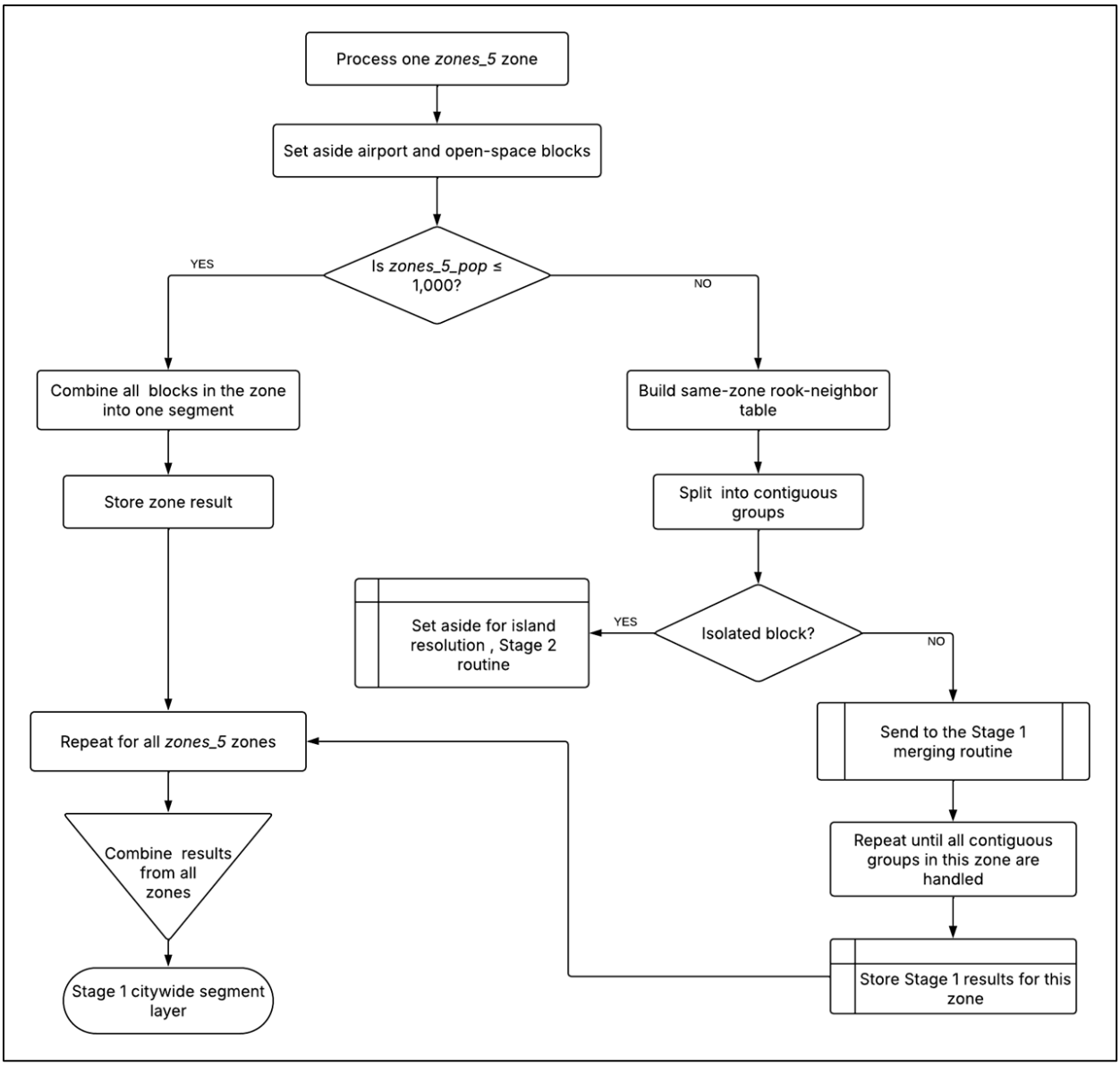


Figure 14. *Workflow diagram for block merging and city segments creation.*

Prepare Inputs

The first notebook creates a new workspace ‘segments_v2’ for all inputs and outputs associated with city-segment creation. Before creating the working block layer, the workflow checks the geometry of the source blocks and zones_5 since invalid polygon geometry can interfere with later spatial joins, boundary measurements, adjacency calculations, and polygon aggregation. Block geometry is repaired on a working copy rather than modifying the source data.

A within polygon centroid is created for every block and spatially joined to the zone layer. The resulting zones_5_ID identifies the zone to which the block belongs, while zones_5_pop stores the population of that zone.

Two binary block attributes, open_space and airport, are created from a composite_class field from the GRID3 block data. The GRID3 block-generation procedure determined the classification; this workflow simply converts those classifications into flags that can be used during merging. Blocks flagged as open space or airport are excluded from the block-merging process and retained independently rather than being combined with ordinary city blocks to meet a population target.

The notebook also calculates block area (block_area_m2) and perimeter (block_perimeter_m). Area is required for the building-density measures used later in the merge criteria, while area and perimeter are used to evaluate the compactness of potential merged segments. The prepared block layer is written to blocks_5.gpkg. Important attributes carried into subsequent notebooks therefore include zones_5_ID, zones_5_pop, open_space, airport, population, block_area_m2, and block_perimeter_m.

The notebook also selects the Overture building polygons associated with the analysis blocks and projects them to the local projected coordinate system for use in the next step.

Calculate Block-Level Building Statistics
The second notebook characterizes the distribution of building sizes and their relationship to block area. These statistics support exploratory analysis of the block dataset and provide measures that can later influence block aggregation. The current merging algorithm uses two building-density measures: building-area density and building-count density—while the other statistics provide additional information for interpretation, diagnostics, and possible future refinements of the merging procedure.

The following attributes are calculated:

Field	Description
bldg_count	Number of buildings assigned to the block
bldg_area_min	Minimum building footprint area among buildings assigned to the block
bldg_area_max	Maximum building footprint area
bldg_area_median	Median building footprint area
bldg_area_stdev	Standard deviation of building footprint area
bldg_area_sum	Total building footprint area lying within the block
bldg_area_density	Building footprint area divided by block area
bldg_count_density	Number of buildings per hectare of block area

Table 4. *Block-level building statistics*

Building count and building-size statistics are based on point assignment and overlap with the block. The full footprint area of that building is then used when calculating the minimum, maximum, median, and standard deviation. This ensures that a building crossing a block boundary is counted as one building rather than being counted in multiple blocks.

Total building footprint within the block (bldg_area_sum) is calculated differently. Building polygons are intersected with block polygons, so only the portion of a footprint lying inside a block contributes to that block's building-area total. Building area density (bldg_area_density), also referred to as block coverage, is therefore calculated as intersected building area divided by block area. Building count density (bldg_count_density), however, expresses the number of buildings per hectare based on point assignment. Considered together, the two density measures describe different aspects of built form: one measures how much of block area is covered by buildings, while the other measures how many buildings there are relative to block area.

During block merging, building count and building area are summed as blocks are combined and the two density measures are recalculated for the growing region. Therefore, during merging, building similarity comparisons reflect the current merged geographies.

Topology Diagnostics

The decision to introduce topology diagnostics was introduced after initial attempts at block merging and review of the citywide block layer created by replacing selected parent blocks from workflow 2 with their subdivided child blocks. That review identified very small geometric discrepancies along certain block boundaries—for example, where child blocks met an unsplit neighboring block or where child blocks originating from different parent blocks met one another. These discrepancies appear to have resulted from small coordinate differences introduced during the tessellation process, particularly affecting the outer block boundary. They were generally on the millimeter or submillimeter scale and did not represent meaningful changes in the intended block geography, but they could affect the adjacency relationships used by the merging algorithm.

This matters because the first stage of block merging depends on a strict definition of contiguous neighbors. Two blocks in the same zone are treated as rook neighbors only when they share more than 0.05 m of polygon boundary. A corner or point contact is not sufficient. A very small gap or overlap caused by coordinate precision can incorrectly remove an intended neighbor relationship or alter the measured length of the shared boundary.

The shared-boundary measurement matters for two reasons. First, it determines which blocks are able to merge with one another. A missing shared edge can split what should be one connected set of blocks into separate groups or make an individual block appear isolated. Second, shared-boundary length enters the geometry calculations used during merging. It affects the estimated perimeter and compactness of a potential merged region and is also used to identify merges connected through unusually narrow shared boundaries.

The topology notebook therefore examines the raw block geometry for point-only contacts, small and larger overlaps, small same-zone gaps, and changes in the resulting rook-neighbor network. It then repeats the adjacency analysis after placing coordinates on precision grids of 0.01 m, 0.05 m, and 0.10 m. For each grid, the workflow evaluates how many rook-neighbor relationships are created or lost, whether the number of connected groups or isolated blocks changes, whether geometries remain valid, and whether polygon areas change appreciably.

The 0.01 m grid increased the number of appropriate rook-neighbor relationships, while increasing the grid size to 0.05 or 0.10 m produced no further improvement. The block-merging notebook therefore uses a 0.01 m working precision. This adjustment is applied to the working geometry rather than the original source data.

The precision adjustment is intended to address millimeter scale coordinate discrepancies, not substantive gaps or overlaps in the block fabric. A future version of the workflow should correct this issue earlier, in workflow 2. In particular, the outer boundary of the tessellation used to create child blocks should be snapped back to the exact parent-block boundary before the children are inserted into the citywide block layer. Doing so would reduce the need for this downstream precision correction.

Block Merging

The fourth notebook performs the actual aggregation. It has two sequential parts. The first forms city segments using blocks that share boundaries within the same zone. Blocks or block groups remaining below the 500-person threshold because they cannot be resolved through contiguous neighbor relationships are addressed in a second step. All contiguous-neighbor merging is completed before the second step begins.

Contiguous-Neighbor, Zone-Constrained Block Merging

The procedure works through the zones one at a time. Because blocks from different zones are never allowed to merge, the result for one zone does not depend on the processing of another zone. The zone-by-zone results are ultimately written together as a citywide segment layer.

Zones with populations of 1,000 or less do not require the merging procedure. After open-space and airport blocks have been set aside, all other blocks in the zone are automatically combined into a single segment. The resulting segment may be multipart if some of its blocks are spatially disconnected.

For zones containing more than 1,000 people, the workflow constructs rook-neighbor networks using the 0.01 m working geometry. Blocks are connected in a network when they belong to the same zone and share more than 0.05 m of boundary.

A contiguous group of rook neighbors consists of blocks that can be reached from one another through a chain of shared-edge relationships. For example, blocks A and C belong to the same group if A shares an edge with B and B shares an edge with C, even if A and C do not touch directly. In graph terminology these groups are connected components—the practical meaning is that there is a set of blocks within which contiguous merging is possible.

A block with no shared-edge neighbor forms a group by itself. Such blocks are set aside rather than forced into a non-contiguous merge during this first step. Similarly, if the combined population of an entire contiguous group is below 500, no sequence of within-group merges can bring that group to meet the threshold. These cases can therefore remain below 500 after the first step and are reconsidered later.

Within each contiguous group, the merging procedure begins by treating every individual block as a separate region. As merging proceeds, a region may therefore represent either one original

block or several blocks that have already been combined. Only regions currently below 500 people initiate a merge. A region that has already reached 500 or more people does not independently seek another neighbor, although it can still receive a neighboring under-500 region.

Choosing which block/region to address first
At each iteration, there may be several under-500 regions requiring attention. At the very beginning, the algorithm gives priority to the block with the fewest remaining options.

It first considers how many blocks could be merged with the target while keeping the resulting population at or below 1,000. The block or region with the fewest such options is addressed first. If several blocks/regions are tied, preference is given to the one with fewer neighbors overall and then to the one with the lower population. Random selection is used only to resolve remaining ties.

This “most constrained first” rule is intended to address difficult cases before an earlier merge elsewhere removes one of their few remaining options.

Evaluating possible merges
Once an under-500 target region has been selected, its current rook neighbors are considered as potential recipients. If at least one neighboring merge would keep the combined population at or below 1,000, candidates that would exceed 1,000 are temporarily excluded. A population above 1,000 is considered only when none of the available neighboring options can remain below that threshold.

Each remaining candidate is evaluated using a composite score. Lower scores indicate more attractive merges:

Score = 5(future-feasibility penalty) + 1.5(building-heterogeneity penalty) + 1(compactness penalty) + 0.15(population penalty) + 0.25(neck penalty) + 2(population-overflow penalty)

The *future-feasibility* term discourages a merge that would leave another nearby under-500 region with few or no reasonable options. This receives the largest coefficient because a locally attractive merge can otherwise make it difficult or impossible to resolve another region later.

The *building-heterogeneity* term compares building-area density and building-count density between the two regions. Differences of 20 percent or less in both measures are treated as practically equivalent and receive no heterogeneity penalty. Larger differences increasingly discourage the merge.

The *compactness* term favors merged polygons with a smaller perimeter relative to their area. It uses the Polsby-Popper measure, which approaches 1 for compact shapes and decreases as polygons become elongated or irregular.

The *population* term favors reaching the 500-person minimum without unnecessarily increasing the population toward the upper end of the target range. Population above 1,000 is handled separately through the overflow term.

The *neck* term discourages joining two regions through a very short shared boundary relative to their perimeter. Such a merge can create a segment in which two larger areas are connected only through a narrow geometric connection.

Finally, the *overflow* term penalizes the amount by which a potential segment would exceed 1,000 people when an over-1,000 merge is unavoidable.

The weights are design parameters rather than estimated coefficients. They determine how strongly these considerations compete with one another. Increasing the heterogeneity weight would make differences in building density more influential and would cause the algorithm to favor morphologically similar neighbors more strongly, potentially accepting a less compact shape or a less convenient population outcome in exchange. Increasing the compactness weight would have the opposite effect: shape would matter more relative to differences in built form. These weights do not change the zone constraint, the exclusion of open-space and airport blocks, or the Stage 1 requirement for shared-edge adjacency.

Figure 15 illustrates how the zone constraint and building similarity term operate together. In the left panel, the green pink, purple, and blue backgrounds identify zones_5 membership; the darker boundaries mark zone edges and the finer gray lines mark blocks. Red hot-spot buildings and blue cold-spot buildings reveal contrasting development patterns. The right panel shows the final city segments. No segment crosses a zone boundary, yet the internal boundaries are not predetermined by the zones: among the possible within-zone alternatives, the scoring process favors groups whose building character is more similar than different.

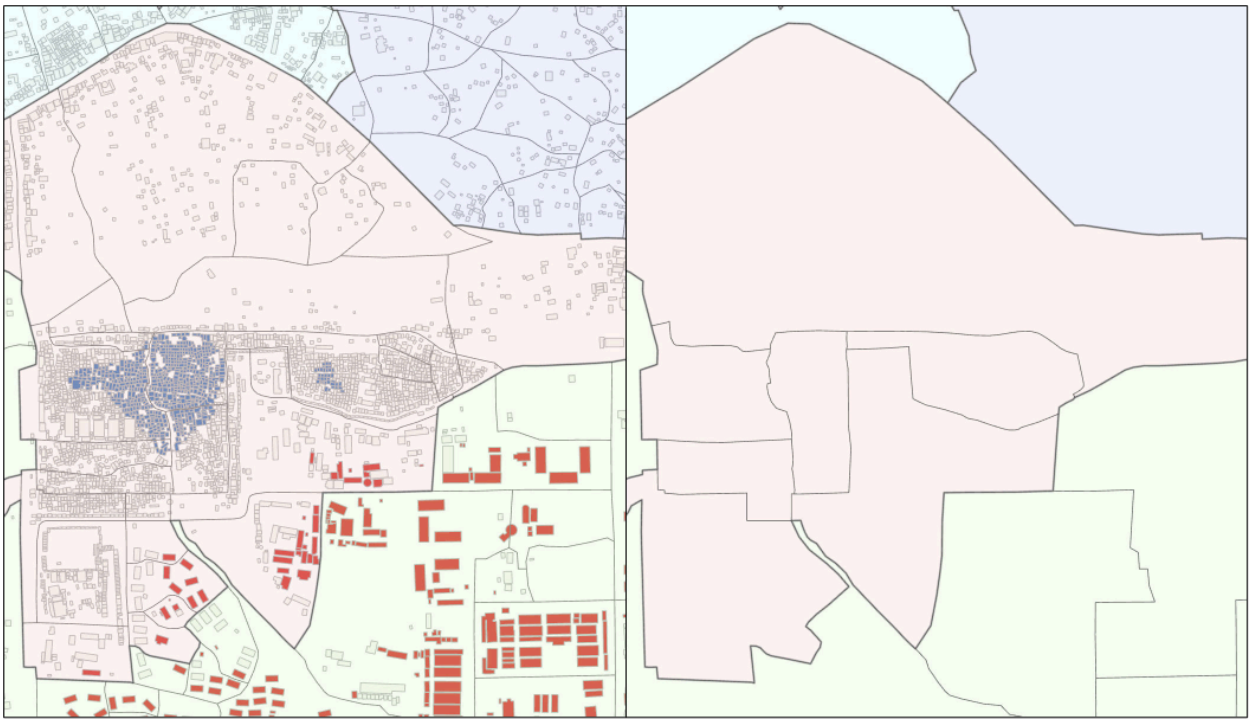


Figure 15. *Zone constrained merging across contrasting building-density patterns.*

Why we repeat the procedure 20 times

An important consideration with this type of aggregation is that every accepted merge changes the nature of the problem that needs to be addressed. After each merge, populations change, building densities are recalculated, perimeter and compactness change, shared-boundary relationships change, and some neighboring options disappear while others become available.

The workflow therefore allows limited variation among nearly equivalent choices. At each iteration it identifies the best candidate score and then forms a small set containing every candidate whose score is within 0.03 score units of that best value. One candidate is randomly selected from this near-best set.

A purely greedy algorithm would always choose the single lowest-scoring merge available at each step. That choice may appear best at the moment it is made but can lead to a poorer outcome several steps later. A different candidate whose score was nearly as good might preserve better options for the rest of the group.

The merging sequence is repeated 20 times for each contiguous group. The random choices use fixed pseudorandom seeds, meaning that the variation is reproducible: running the same data with the same parameters reproduces the same 20 alternatives. The code therefore adopts a seeded near-greedy search—'near-greedy' because choices are restricted to candidates very close to the current best score, and 'seeded' because the choices can be reproduced.

The important comparison occurs after each complete run has finished. After all 20 runs are complete, the workflow ranks the 20 alternative segment configurations according to a fixed hierarchy of outcome criteria.

The first priority is to minimize the number of segments remaining below 500 people. If runs are tied on that measure, the workflow next minimizes the total population deficit below 500. It then considers population overflow above 1,000, the maximum segment population, compactness, and finally mean building heterogeneity. Lower-priority criteria are considered only after higher-priority criteria are tied.

This creates an important distinction between two parts of the method. The candidate score answers, "Which of the available merges is attractive at this particular step?" The completed-run comparison answers, "Which full sequence of merge decisions produced the strongest overall segmentation?" Evaluating multiple complete sequences reduces the likelihood that a single locally attractive early decision determines the final geography.

Once the best run has been selected for each contiguous group, its block assignments are retained. After all zones have been processed, these assignments are combined into the citywide Stage 1 segment layer.

Resolving Low-Population Islands

The second part of the merging notebook begins only after contiguous-neighbor merging has been completed for the entire city. It works with the Stage 1 segments as whole units and does not reopen their internal block assignments.

The principal targets are under-500 segments that could not be resolved through contiguous merging. These include isolated blocks with no shared-edge neighbor, contiguous groups whose combined population was itself below 500, and occasional under-500 remainders produced by the iterative Stage 1 process. These cases are treated together as low-population island or unresolved segments.

Unlike Stage 1, this step cannot rely on shared boundaries, because the problem being addressed is precisely that the target segment lacks an appropriate contiguous route to another segment. For each under-500 target, the workflow therefore considers up to the eight nearest eligible Stage 1 segments within the same zone, measured by polygon-to-polygon distance.

As in the first stage, candidates that keep the combined population at or below 1,000 are preferred. An over-1,000 attachment is considered only when none of the nearby candidates can remain within that threshold.

Targets with the fewest preferred recipients are addressed first. Ties are then resolved by considering the total number of nearby candidates, distance from the nearest candidate, population, and finally a reproducible random tie-breaker. The intention is to again deal first with cases that have the fewest remaining options.

The Stage 2 candidate score uses future feasibility, attachment distance, building heterogeneity, population, and overflow:

Score = 5(future-feasibility penalty) + 2(distance penalty) + 1.5(building-heterogeneity penalty) + 0.15(population penalty) + 2(population-overflow penalty)

Compactness and neck penalties are not used in this stage because the target and recipient need not share a boundary. Instead, distance discourages unnecessarily long attachments. Building-density similarity continues to favor combinations with comparable built form.

As in Stage 1, the algorithm does not rely on one sequence of decisions. Twenty reproducible runs are performed for each zone containing Stage 2 targets, with selection among candidates whose scores lie within 0.03 of the current best score. The completed runs are then compared. Preference is given first to the run leaving the fewest unresolved under-500 segments and the smallest remaining population deficit, followed by reduced population overflow, shorter attachment distances, lower building heterogeneity, and lower maximum population.

The selected attachments are dissolved into the final city-segment geography. Because some Stage 2 attachments connect previously disconnected polygons, the resulting segment may be multipart.

Final Outputs and Quality Assurance
Stage 1 produces a citywide segment layer and a crosswalk assigning each source block to its Stage 1 segment. Stage 2 produces the principal final city-segment layer, segments_islands_resolved, together with blocks_with_final_segment_id, an attachment-line layer, and a crosswalk between Stage 1 and final segment identifiers.

The workflow also retains run summaries, merge logs, candidate diagnostics, zone summaries, and segment-level statistics so that individual decisions and alternative runs can be reviewed. Population totals are checked citywide and within each zone to confirm that the process changes how blocks are grouped but does not change the total population represented. Table 5 summarizes the final reduction from blocks to city segments for Juba, Kigali, and Johannesburg.

Citywide count	Juba	Kigali	Johannesburg
Blocks (Final, after splitting)	19,668	16,620	59,034
Segments	1,890	3,276	7,399
Blocks per segment	10.4	5.1	8.0

Table 5. *Final block and city-segment counts by city.*

Segment Attributes

Field name	Description	Source
fid	unique feature id	native identifier
segment_area_m2	block area in square meters	calculated
segment_perim_m	block perimeter in meters	calculated
building_count	the count of buildings whose centroids fall within the block (where the centroid has to be constrained to lie inside the building polygon)	Overture, spatial join (all building related fields use Overture buildings) https://docs.overturemaps.org/guides/buildings
total_building_area	the sum of all building polygon area that lies within the block boundary. Note that can include buildings whose centroids fall outside of the block border, but have some building area within the block boundary	calculated
min_building_area	the minimum building polygon area (smallest building) where the entirety of that building's polygon falls inside the block boundary	calculated
max_building_area	the maximum building polygon area (largest building) inside the block boundary. In this case, the entirety of the building does not have to be within the block boundary.	calculated
median_building_area	the median area of fully contained buildings within the block	calculated
average_building_area	the average area of fully contained buildings within the block	calculated
stdev_building_area	the standard deviation of fully contained buildings in the block	calculated
building_area_density	also known as block coverage, it is the total building area divided by the block area	calculated
building_count_density	This is the number of features per unit area, in this case: building count divided by block area in hectares	calculated

Field name	Description	Source
block_count	The number of blocks merged together to create the segment	calculated
zones_5_ID	the final zone ID associated with blocks that constrains merge candidates	calculated
open_space	1 = open space, 0 = not-open space, based on GRID3 composite class score	GRID3
airport	1 = airport, 0 = not airport, based on GRID3 composite class score	GRID3
pop_lsm_ambient	LandScan Mosaic (2025) Ambient Population aggregated from blocks, which were derived from 100m grids using dasymetric weighting based on building area	ORNL https://landscan.ornl.gov/
pop_lsm_nighttime	LandScan Mosaic (2026) Nighttime Population aggregated from blocks, which were derived from 100m grids using dasymetric weighting based on building area	ORNL https://landscan.ornl.gov/
pop_worldpop	WorldPop-Global 2 for 2025 (R2025A v1) population estimates aggregated from blocks which were derived from 100m grids using dasymetric weighting based on building area	WorldPop https://hub.worldpop.org/
pop_ghspop	GHS-POP for 2025 (R2023A) population estimates aggregate aggregated from blocks which were derived from 100m grids using dasymetric weighting based on building area	EC-JRC https://doi.org/10.2905/2FF68A52-5B5B-4A22-8F40-C41DA8332CFE

6. Limitations and improvements for the future

The current workflow builds blocks and segments only within GRID3 settlement extents. This represents where people live well, but it creates two problems when blocks are aggregated into segments. First are disconnected settlement "islands" because the settlement extents themselves include isolated patches, often with only one or a few buildings. They share no edge with neighbouring blocks, so a separate distance-based step attached them after contiguous merging was finished. As a result, some of the resulting segments are multipart, cover unexpectedly large areas, and are less intuitive than nearby alternatives. For very small patches, the building-similarity and population-balance criteria in the merge score have too little information to give meaningful guidance. Second, the parameters for resolving islands were tuned mainly on Juba. Given that settlement patterns vary widely between cities, these parameters may not generalize.

Recommended direction for full AOI coverage from the start
Future versions should build segments across the full city Area of Interest. Where physical road and river boundaries do not exist, further research and development into other landscape feature delineation might be tested, and as a fall back, a regular geometry like hexgrids could be built into the workflow from the beginning. An improved workflow might look like:

- Start with complete coverage. Tessellate the entire AOI, using GRID3 blocks where settlement exists and a hexagonal grid to fill the remaining area. Every location then belongs to a feature, and no settlement patch is isolated.
- Assign attributes consistently. Give hex grids building counts, population, morphology metrics, and zone identifiers using the same procedures applied to blocks.
- Aggregate through shared edges only. Hex grids provide spatial contiguity, so former settled “islands” and hex grids can be absorbed into contiguous segments during normal merging. Merging can still prioritize the original GRID3 blocks.

Open design questions. Several choices still need to be resolved including how to handle zero- or very-low-population hex grids, for example by allowing peripheral segments to fall below the population target; how to set the hex grid size relative to typical block size; and how to treat very small fragments created where roads, rivers, and zone boundaries intersect.